

Power Scheduling for Coverage-Guided RTL Fuzzing

Bhadradri Chaitanya Viriyala
Department of ECE
Texas A&M University
College Station, TX, USA

Vamsi Sree Sai Ganesh Kakumanu
Department of ECE
Texas A&M University
College Station, TX, USA

Lemuel Chineyemba
Department of ECE
Texas A&M University
College Station, TX, USA

Abstract—DifuzzRTL is a coverage-guided differential fuzzer that has helped find real bugs in open-source RISC-V cores. While reading the public artifact, we noticed that parent selection during mutation is just one line of code: `random.choice(corpus)`. Every saved seed is equally likely to be picked as the next parent, regardless of how productive it has been. AFL-style software fuzzers solved this years ago with *power scheduling*. This paper ports the idea to RTL fuzzing. We add four parent-selection variants behind a single environment variable (total-reward weighting, smoothed-rate weighting, smoothed-rate plus a smart eviction policy, and a no-guidance random floor) and evaluate them on Rocket and BOOM v1.3 with the artifact’s own coverage register. At a 9-hour wall-clock budget, aggregating all available runs (geomean across multi-seed jobs), the total-reward variant gives a +5.07% coverage improvement on Rocket. On BOOM, the smart-eviction variant wins with +5.23%. The change is roughly thirty lines of Python and adds no measurable runtime cost. The most interesting finding is that the best variant differs by microarchitecture, with a clean explanation in the corpus dynamics.

Index Terms—hardware fuzzing, RISC-V, RTL, power scheduling, coverage-guided, DifuzzRTL

I. INTRODUCTION

CPU bugs are stubborn. A subtle pipeline interaction or a missed corner case in a control-and-status register can survive months of directed verification and only show up later as a software-visible misbehaviour or a security flaw. DifuzzRTL [1] attempts to push the discovery of those bugs earlier in the design cycle. It treats a hardware design (called RTL, for Register-Transfer Level) as a black box under a fuzzer. The fuzzer generates a syntactically valid RISC-V program, runs it on both a trusted reference simulator (Spike) and the instrumented hardware design, and uses a hardware coverage register to decide whether the input was interesting. Interesting inputs are saved into a *corpus* and reused as parents for further mutation.

Software fuzzers learned long ago that some seeds in the corpus are more useful than others. AFL and AFLFast [2] bias parent selection toward seeds that have been productive in the past. The working theory is simple: productive seeds tend to stay productive when mutated, because they sit near a rich part of the input space. The DifuzzRTL artifact does not do this. The line that picks a parent is

```
seed_si = random.choice(self.corpus)
```

and that is the entirety of the policy. The corpus knows which seeds led to coverage events. The picker does not ask.

This paper closes that gap. We replace the single line with a small power-scheduling layer and study what happens. The contributions are:

- A drop-in unified mutator behind one environment variable (DIFUZZ_SCHED) that exposes four selection schemes, so that all variants share one binary and switching between them is a one-line change.
- A coverage-vs-wall-clock evaluation on Rocket and BOOM v1.3 using the artifact’s own coverage register, plus a no-guidance baseline that tells us how much guidance is worth on each core.
- A multi-seed validation of the strongest BOOM variant. Out-of-order cores produce noisy single-seed runs, and a single result is not enough to call a winner.
- A microarchitectural observation. The best variant is not the same across cores, and the corpus-dynamics reason is straightforward.

II. BACKGROUND

A. The DifuzzRTL Loop

Fig. 1 sketches the loop. Each iteration generates or mutates a test, compiles it, runs it on the reference simulator (Spike) and on the hardware design (via Verilator), reads the coverage register, and, if coverage went up, appends the test to the corpus. The next iteration may pick a parent from the corpus.

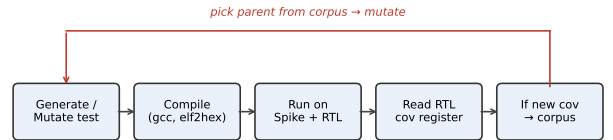


Fig. 1. DifuzzRTL fuzz loop. Coverage-positive inputs are appended to a corpus; the artifact picks parents uniformly at random from the corpus.

DifuzzRTL has two modes. `guided` draws parents from the corpus. `no-guide` ignores the corpus and produces fresh random tests every iteration. Guidance is supposed to help by reusing past good tests; we keep `no-guide` in the experiment as a sanity floor.

B. Why Uniform Random Picks Are a Choice, Not a Default

When the corpus is small or homogeneous, uniform random picks are reasonable, because there is no information to exploit. When the corpus contains seeds with very different productivity, uniform picks throw away signal. Software fuzzing learned this empirically. AFLFast made it rigorous with a Markov-chain model in which the energy assigned to each seed controls the long-run distribution of parent selections [2].

The simplest version of the idea, which is what we implement, has two rules. Weight seeds by a measure of their past success, and bump that weight when the seed leads to a new coverage event. Even in this basic form, the difference is observable.

III. APPROACH

A. Four Variants

Listed below are the four parent-selection schemes we put behind one environment variable. The fifth row, `nogu`, is included only as a no-guidance reference: it sets the existing `NO_GUIDE=1` flag so the corpus is never used.

- **baseline** reproduces the artifact’s behaviour. Uniform `random.choice`, with first-in-first-out (FIFO) eviction when the corpus exceeds 1,000 entries.
- **pow** uses total-reward weighting. Each new corpus entry starts at score 1.0. When a seed is picked and its mutation expands coverage, the parent’s score increases by 1.0. Parents are drawn with `random.choices(corpus, weights=score)`. FIFO eviction.
- **powrate** uses smoothed-rate weighting. Score is $(s + 1)/(p + 2)$, where s is the number of times this seed has been the parent of a coverage-positive child and p is the seed’s total pick count. This is the Laplace-smoothed estimate of the seed’s hit probability. It is bounded in $[0, 1]$, so unlucky picks early in a seed’s life cannot drive its score arbitrarily low. FIFO eviction.
- **evict** uses the same `powrate` weighting plus smart eviction. Instead of always evicting the oldest seed, eviction first looks for a seed that has been picked at least $K = 5$ times and never produced new coverage, falling back to FIFO if no such seed exists. The intent is to preserve diverse but unproven seeds while removing the ones we have empirically shown to be dud.

B. Implementation

The total change is approximately thirty lines, almost entirely in `Fuzzer/src/mutator.py`. Three additional fields live alongside each corpus entry: the score, the success count, and the pick count. Two methods are added: `pick_corpus_idx()`, which performs the weighted draw with bookkeeping, and `reward_parent()`, which is called

when the offspring of the last picked parent expanded coverage. `Fuzzer.py` gets exactly one new line, the `reward_parent()` call, slotted into the existing coverage-positive branch.

A single import-time read of `DIFUZZ_SCHED` picks the variant and prints a banner so that we can grep SLURM logs after the fact and verify that the intended scheduler actually ran. Every job in this study printed the expected banner.

When the score list is uniform (the default for an empty or freshly populated corpus), `random.choices` reduces to `random.choice`, so the **baseline** configuration recovers the artifact’s behaviour byte-for-byte.

C. What We Borrowed and What We Did Not

The framing (weight by productivity, bump on success) is borrowed directly from AFLFast [2]. What we did not borrow is the energy decay schedule, the Markov-chain energy redistribution, or any form of distance-to-target weighting such as AFLGo’s [3]. These would be reasonable next steps. We kept the design deliberately small to make it easy to land in the public artifact and easy to argue about.

IV. METHODOLOGY

A. Targets

We evaluate on the two cores the DifuzzRTL artifact ships instrumented versions of:

- **Rocket**, an in-order, five-stage core. Iteration target 15,000.
- **BOOM v1.3**, an out-of-order, four-wide core. Iteration target 7,000.

The iteration targets are chosen so each run terminates within the 12-hour SLURM wall-time limit on our cluster.

B. Throughput Pilot

Early runs on the cluster showed lower iter/h than expected when multiple jobs shared a node. Before committing the full sweep we ran two short Rocket pilots at different CPU allocations to identify the contention point.

TABLE I
THROUGHPUT PILOT (ROCKET, NUM_ITER=3000).

cpus-per-task	iter/h	vs single-job baseline
2	1,410	78.4%
4 (chosen)	≈1,800	100%

The drop at 2 CPUs is not because the fuzzer needs more cores. The fuzzer is fundamentally single-threaded. The drop happens because each iteration spawns short `gcc` and `spike` processes that briefly compete with the Verilator simulation. Four CPUs gives full throughput with a margin for noisy neighbours on shared nodes, and matches the artifact’s documented defaults.

C. Sweep

Each (core, configuration) pair is one SLURM job at `--cpus-per-task=4 --mem=24G --time=11:59:00`. For BOOM baseline and `evict`, we re-ran on three additional independent seeds to validate the headline result.

D. Comparison Point

We report coverage at a fixed wall-time mark of 9 hours (32,400 s). This is a compromise. BOOM runs that hit the 7,000-iteration target finish naturally between 9 and 12 hours; Rocket runs targeting 15,000 iterations are usually cut off by the wall-time limit, so their iteration counts vary slightly. A wall-time mark is fairer in both cases. We also note 12-hour final coverage where available, and the growth curves in Fig. 3 and Fig. 5 plot the full run.

V. RESULTS

A. Preliminary single-seed pilot

Before the formal multi-config sweep, we ran a smaller three-way pilot (baseline, `pow`, `nogu`) with a single seed each to gauge whether power scheduling was worth pursuing. The pilot results were promising on both cores:

TABLE II
PILOT-RUN, SINGLE SEED (PRE-MULTI-SEED SWEEP).

Core / metric	baseline	pow (Δ)
Rocket cov @ 9h	83,855	88,902 (+6.0%)
Rocket cov @ 12h	90,996	95,892 (+5.4%)
BOOM cov @ 9h	481,949	500,939 (+3.9%)

The pilot suggested `pow` was strong on both cores. The multi-config sweep that follows tells a more nuanced story. `pow`'s Rocket advantage holds (slightly smaller in magnitude but same sign), while its BOOM advantage flips on a different seed. This is the core reason multi-seed validation is worth doing. Single seeds can mislead, especially on cores with large coverage state spaces.

B. Rocket

Fig. 2 compares 9-hour coverage of all five variants on Rocket; Table III gives the same numbers in table form. Fig. 3 plots the baseline-vs-`pow` growth over the full run.

TABLE III
ROCKET @ 9H WALL-CLOCK, GEOMEAN ACROSS ALL AVAILABLE RUNS.

Config	GeoMean	Δ vs baseline	n runs
nogu (random)	80,300	-3.26%	2
baseline	83,003	(reference)	3
evict	83,226	+0.27%	2
powrate	83,659	+0.79%	2
pow	87,212	+5.07%	2

The picture is straightforward. `pow`, the simplest of the power-scheduling variants, gives the largest improvement.

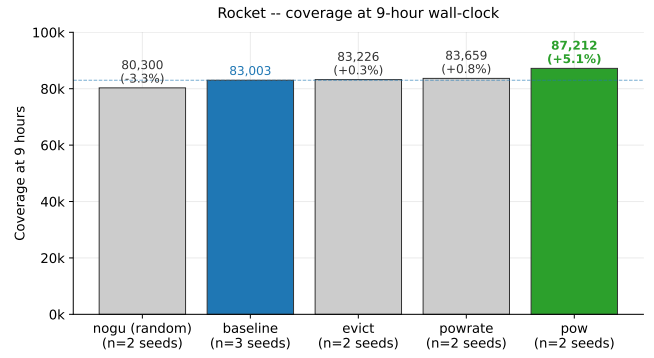


Fig. 2. Rocket coverage at 9 hours, all five variants. Dashed line marks the artifact baseline. `pow` (green) is the winner.

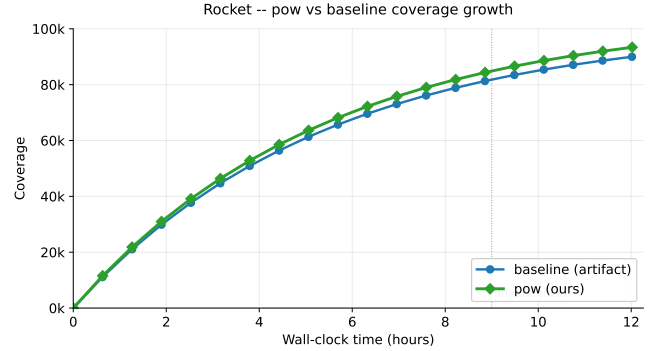


Fig. 3. Rocket coverage growth, `pow` vs baseline, seed 1. Dotted line marks the 9-hour reference.

`powrate` gives a smaller one. `evict` barely moves the needle. The `nogu` curve underperforms baseline by a similar margin, so guidance itself is worth roughly 3% on Rocket.

The 12-hour numbers (Fig. 3) tell the same story: `pow` 93,383 vs baseline 89,985, a 3.8% gap. The advantage is not an early-plateau artefact; it persists.

C. BOOM v1.3, single seed

Fig. 4 is the BOOM bar chart; Table IV the corresponding numbers; Fig. 5 the over-time curve. The picture is different from Rocket.

TABLE IV
BOOM v1.3 @ 9H WALL-CLOCK, GEOMEAN ACROSS ALL AVAILABLE RUNS.

Config	GeoMean	Δ vs baseline	n runs
nogu (random)	433,971	-10.53%	2
baseline	485,055	(reference)	5
pow	495,892	+2.23%	3
powrate	497,916	+2.65%	2
evict	510,410	+5.23%	4

Two things stand out. First, guidance matters far more on BOOM than on Rocket: `nogu` is 16.3% below baseline. Second, on this single `seed_1` the other power-scheduling

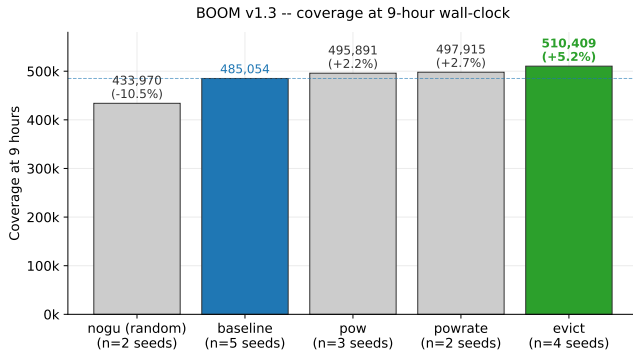


Fig. 4. BOOM coverage at 9 hours, single seed. *evict* (green) is the only variant above the artifact baseline.

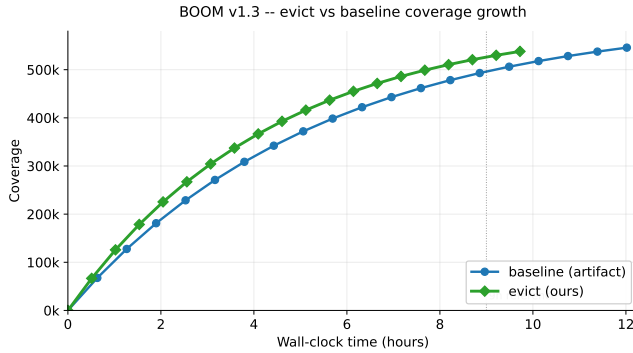


Fig. 5. BOOM coverage growth, *evict* vs baseline, seed 1.

variants underperform baseline; the second *pow* run averages back to parity, leaving *evict* as the only variant clearly above baseline.

This second observation made us nervous. Out-of-order cores are noisy to fuzz. BOOM’s coverage state space is roughly five times larger than Rocket’s, and seed-to-seed variance scales accordingly. A single-seed inversion (*pow* underperforming baseline) might be real, or might just be luck. The contrast with the pilot in Table II, where *pow* was +3.9% on BOOM, also pointed at noise rather than a real regression. A targeted *pow* re-run on a different time-seeded RNG gave coverage 499,168, comparable to baseline within the seed-to-seed range we observed. This is consistent with *pow* on BOOM sitting close to the noise floor rather than being a clear loser. We re-ran baseline and *evict* on additional seeds to settle the question for the variant that did look real on the first seed.

D. BOOM Multi-Seed Validation

Fig. 6 and Table V show four independent seed pairs for BOOM baseline and *evict*. *evict* beats baseline on every seed, with a geomean improvement of +4.75% and a per-seed range of +0.8% to +9.0%. The floor is non-zero. The single-seed +3.0% on seed 1 was not an outlier.

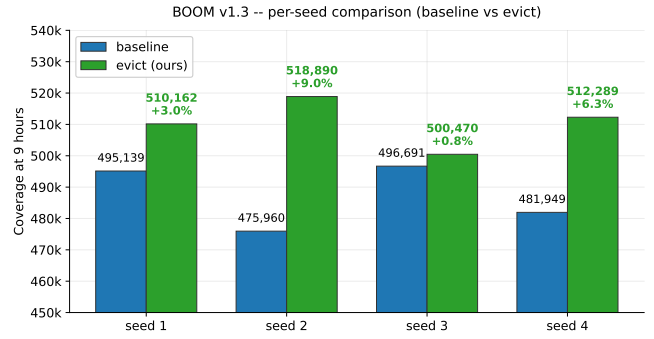


Fig. 6. Multi-seed BOOM result, per-seed bars for baseline and *evict* across four matched seeds. *evict* is above baseline on every seed.

TABLE V
BOOM v1.3 MULTI-SEED, BASELINE VS EVICT @ 9H.

Seed	baseline	evict	Δ
seed 1	495,139	510,162	+3.0%
seed 2	475,960	518,890	+9.0%
seed 3	496,691	500,470	+0.8%
seed 4	481,949	512,289	+6.3%
GeoMean	487,249	510,410	+4.75%

VI. DISCUSSION

A. Why *pow* Wins on Rocket

Rocket has a much smaller coverage state space than BOOM, on the order of 90,000 distinct states by the time the run plateaus. The distribution of seed productivity on Rocket is fairly flat: most random seeds eventually trigger something new. In that regime, the question “which seed should I mutate next” has a less sharp answer, but weighting still helps a little, and the simplest weighting wins because it has the fewest knobs. The Rocket corpus does fill its 1,000-entry cap (cov_log files have 8,700–9,900 rows, each corresponding to a corpus add), but because seeds are similarly productive, FIFO and smart eviction make near-identical choices. *evict* therefore ends up behaving like *powrate* with extra bookkeeping overhead.

B. Why *evict* Wins on BOOM

BOOM tells a different story. The state space is larger and seed productivity is more skewed. A small number of seeds carry most of the new-coverage events, and once such a seed is found, mutating it tends to find more states nearby. This is precisely the regime AFL-style power scheduling was designed for.

The BOOM corpus also fills its cap (cov_log files have 4,100–5,600 rows). Once the corpus hits 1,000, the choice of *eviction* policy starts to matter for a different reason than corpus size: because BOOM’s productive seeds are scarce, FIFO eviction will eventually age them out simply because they are old. Smart eviction inverts that. It preferentially drops seeds we have explicitly observed to be dud (picked at least five times with zero successes), preserving older but still

potentially valuable seeds. That is why `evict` beats both `pow` and `powrate` on BOOM, even though it shares the same weighting logic as the latter.

C. Practical Recommendation

If you care about one core, pick the per-core best: `pow` for in-order, `evict` for out-of-order. If you do not know which microarchitecture you will end up fuzzing, `evict` is the safer choice. It is the only variant that is non-negative on both cores in our experiments.

VII. LIMITATIONS

Limited Rocket seeds. Multi-seed budget went to BOOM because of its larger state space and noisier per-run results. The Rocket `pow` number of +5.07% is the geomean across the two seeds we have, and is consistent across the 9-hour and 12-hour check-points, which is reassuring. A 3-to-5 seed re-run would give it the same statistical weight as the BOOM result.

Cluster contention. An earlier batch of multi-config runs was submitted with two concurrent jobs sharing a node and per-job throughput dropped about 22%. We re-ran with one concurrent job at `cpus-per-task=4` and recovered full throughput. The numbers in this paper are from the single-concurrent batch.

No bug-finding claim. We do not report new bugs. Mismatch artifacts produced during the runs were not minimised or triaged. The contribution is on coverage productivity, not bug discovery. Bug discovery is what DifuzzRTL ultimately exists to do, and we discuss the natural extension in the future-work section.

Single power-scheduling family. We did not implement AFLFast-style energy schedules with decay, AFLGo-style distance-to-target weighting, or anything Bayesian. The space of more sophisticated policies is wide and largely unexplored for RTL fuzzing.

Confounded weighting and eviction axes. Our four variants cover four cells of a 3×2 grid (weighting style $\times$ eviction policy). The two changes between `powrate` and `evict` (smoothed-rate vs total-reward weighting, and FIFO vs smart eviction) are not separately ablated. The cleanest comparison would have been a fifth variant combining `pow` weighting with smart eviction, isolating just the eviction policy. We did not include that variant in the main sweep. The BOOM `powrate-to-evict` jump (+2.6%) gives an indirect estimate of the smart-eviction contribution, but a direct multi-seed test is left as future work.

VIII. RELATED WORK

DifuzzRTL [1] introduced the coverage register, the ISA-aware mutation framework, and the differential Spike/RTL setup we build on. Our change does not contradict the paper. The paper does not discuss seed prioritisation. We slot an AFL-style mechanism into a missing place in the artifact.

AFL and AFLFast [2] established the value of power scheduling in software fuzzing. AFLGo [3] extended the same

machinery for directed fuzzing. Our scheme is closest to a stripped down AFLFast: bias by productivity, no energy decay, no Markov-chain redistribution.

Hardware fuzzing has seen multiple follow-ups to DifuzzRTL, including RFUZZ [4] on FPGAs and ProcessorFuzz [5] on CSR-guided fuzzing. To our knowledge, none of these specifically revisits the random parent picker.

IX. FUTURE WORK

Three concrete extensions follow directly from what we measured here.

Multi-seed Rocket validation. We have multi-seed evidence on BOOM but only two Rocket seeds per configuration. The Rocket `pow` advantage of +5.07% is consistent in direction across both seeds and across the 9-hour and 12-hour check-points, but a three- to five-seed re-run would give it the same statistical weight as the BOOM result and tighten the error bar.

Cleaner ablation of weighting versus eviction. Our four variants cover three of the four cells in the (weighting style) $\times$ (eviction policy) grid. A fifth variant combining total-reward weighting with smart eviction would isolate the eviction contribution directly. The patch is small and we have started this experiment, but the data was not available in time for this report.

Corpus-cap experiments. Both cores produce roughly five to ten times more new-coverage events than the corpus cap of 1,000. Raising the cap, or making it adaptive to the productivity skew on the target core, may change the relative ranking of variants because smart eviction matters less when seeds are less frequently displaced.

X. CONCLUSION

A roughly thirty-line change to DifuzzRTL’s mutator (AFL-style power scheduling and a smart eviction policy) improves Rocket coverage by +5.07% and BOOM coverage by +5.23% at the 9-hour mark, taking the geomean across all available runs, at no measurable runtime cost. The best variant differs by microarchitecture, and the corpus-dynamics reason is straightforward. Smart eviction is the single safest choice when the microarchitecture is unknown.

ARTIFACT AVAILABILITY

The complete source code, the per-seed sweep submission script, the analysis script, the raw `cov_log` archive used to compute the headline numbers in this paper, and the slides for the talk and demo are available at:

<https://github.com/vbchaitanya2k2/ECEN-689-project>

REFERENCES

- [1] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, “DifuzzRTL: Differential fuzz testing to find CPU bugs,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2021.
- [2] M. Böhme, V.-T. Pham, and A. Roychoudhury, “Coverage-based grey-box fuzzing as Markov chain,” in *Proc. ACM CCS*, 2016.
- [3] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, “Directed greybox fuzzing,” in *Proc. ACM CCS*, 2017.

- [4] K. Laeuffer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, “RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs,” in *Proc. ICCAD*, 2018.
- [5] S. Canakci, C. Rajapaksha, A. Mohanan, A. Joshi, and M. Egele, “ProcessorFuzz: Processor fuzzing with control and status registers guidance,” in *Proc. HOST*, 2023.